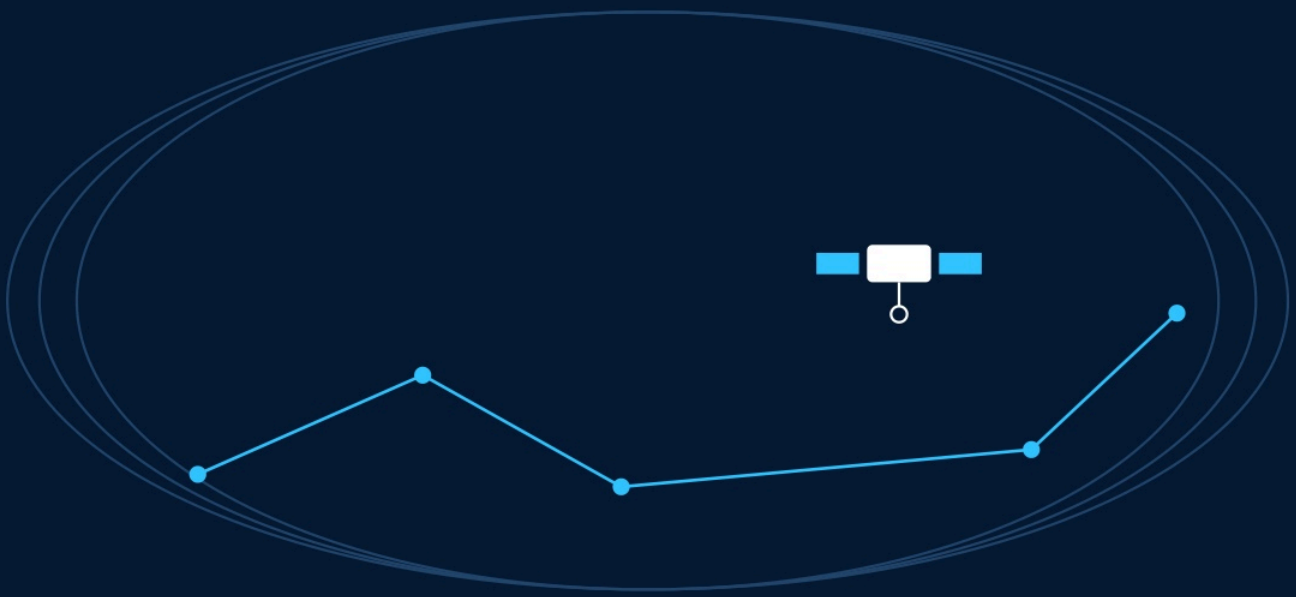


ORBITLINK SATELLITE INTERNET NETWORK

SQL Database Management & Analytics Project



Prepared by Prativa Baliarsingh	Technology MySQL / SQL	Portfolio GitHub / LinkedIn placeholder
Project type Relational database + analytics	Source scope 6 tables • 20,000 records	Date 1 September 2026

EXECUTIVE SUMMARY

OrbitLink Satellite Internet Network • SQL Database Management & Analytics

20,000

Total records

6

Relational tables

18,200

Usage sessions

700

Customers

Project at a Glance

OrbitLink is an end-to-end SQL database and analytics case study for a satellite internet network. The project models customers, subscriptions, satellites, maintenance activity, ground stations, and individual usage sessions in a relational MySQL database. It demonstrates how operational data can be converted into reliable reporting, performance monitoring, financial analysis, and data-quality controls.

Business Value

The database supports four practical stakeholder perspectives: network engineering can monitor satellite performance and latency; maintenance teams can track incidents and downtime; billing and finance can analyze subscription revenue; and customer support can investigate customer activity and recent usage.

Analytical Scope

The project includes 75 real-world business scenarios covering customer and subscription analytics, satellite operations, network performance, usage behavior, revenue, maintenance reliability, data quality, query optimization, reporting views, and access control. Advanced examples include LAG/LEAD, RANK/DENSE_RANK, ROW_NUMBER, NTILE, moving averages, CTEs, and multi-table analytical joins.

Professional Positioning

This project demonstrates more than SQL syntax. It shows a complete analytical workflow: understanding a business problem, designing a relational model, enforcing data integrity, loading operational data, answering business questions, optimizing recurring access paths, applying database privileges, and communicating results through KPIs and visual analysis.

1. Executive Summary

OrbitLink is presented as a satellite internet network using a low-orbit satellite constellation and ground-station infrastructure to deliver broadband service to remote and underserved regions. The operational database captures the network entities and the individual internet sessions generated by customers. The project brief states that the database is used by network engineering, billing and finance, customer support, and satellite maintenance teams.

The supplied SQL implementation contains six connected tables—Satellites, GroundStations, Customers, Subscriptions, MaintenanceRecords, and UsageLogs—and exactly 20,000 inserted records across those tables. UsageLogs is the largest table, with 18,200 session records in the supplied dataset.

From a Data Analyst / SQL Developer perspective, the project is valuable because it combines database design with practical questions: subscription activity, satellite utilization, latency, data consumption, revenue, maintenance downtime, data-quality controls, indexing, reusable views, and role-based access. The 75 scenarios deliberately move from straightforward SELECT statements to CTEs, subqueries, window functions, constraints, views, indexing, and security operations.

Source accuracy: numerical findings in this report are calculated from the supplied INSERT dataset; implementation observations are based on the supplied SQL and EER model. No external business assumptions are used.

2. Project Overview

Business context

The project scenario describes an operational environment where satellite status, ground-station capacity, customer subscriptions, maintenance events, and usage sessions are continuously recorded. The database is intended to make operational signals visible before they become larger business problems.

What the database manages

- Satellite fleet and status
- Ground-station locations and capacity
- Customer identity and geography
- Subscription plans, fees, start dates, and status
- Maintenance incidents and downtime
- Individual usage sessions, data consumption, duration, and latency

Project objectives

- Build a relational SQL database for a satellite internet network.
- Manage satellite and ground-station information.
- Track customers and subscriptions.
- Monitor maintenance activity and downtime.
- Store and analyze individual internet sessions.
- Identify network performance and utilization issues.
- Analyze customer engagement and usage patterns.
- Monitor revenue and subscription plans.
- Support data integrity, query performance, and access control.

3. Business Problem / Real-World Scenario

A satellite internet operator needs a centralized source of truth because the operational picture is distributed across fleet assets, service relationships, customer activity, and network telemetry. A relational model allows each operational domain to be stored once and joined when an analysis needs the wider picture.

The scenarios in the supplied project show how SQL can transform raw records into operational decisions—for example, identifying subscriptions with little activity, comparing latency across satellites, calculating revenue by plan, finding high-incident satellites, cleaning imported text, and creating reusable reporting views.

The source project description explicitly frames the database as a live operational system supporting engineers,

4. Technology Stack

Technology	Purpose
MySQL / SQL	Database creation, DDL, DML, analytical queries
SQL Joins	Combining customers, subscriptions, satellites, maintenance, and usage
Aggregate functions	COUNT, SUM, AVG, MIN, MAX
Window functions	LAG, LEAD, ROW_NUMBER, RANK, DENSE_RANK, NTILE
CTEs / subqueries	Intermediate calculations and multi-step analysis
CASE / date / string functions	Classification, time analysis, and text cleaning
Constraints	Primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT, CHECK
Indexes	Single-column and composite search optimization
Views	Reusable reporting datasets
Roles / privileges	Billing and support access patterns; privilege revocation

5. Database Architecture

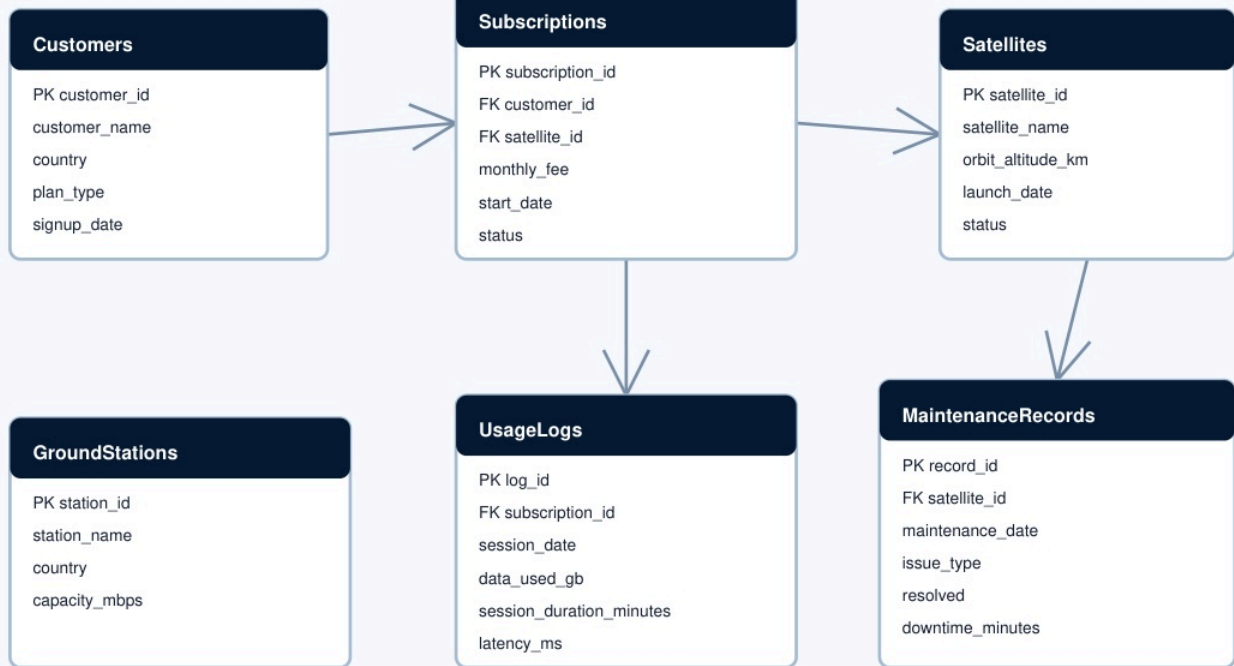
The relational architecture consists of six tables. The SQL source defines primary keys for every table. Subscriptions connects customers to satellites; MaintenanceRecords connects maintenance activity to satellites; UsageLogs connects session telemetry to subscriptions. GroundStations stores station geography and capacity and is used by country-matching scenarios rather than through a declared foreign key.

Table	Rows	Primary key	Business role
Satellites	40	satellite_id	Fleet assets; altitude, launch date, status
GroundStations	25	station_id	Station geography and Mbps capacity
Customers	700	customer_id	Customer identity, country, plan type, signup date
Subscriptions	700	subscription_id	Customer service relationship and satellite assignment
MaintenanceRecords	335	record_id	Maintenance incidents and downtime
UsageLogs	18,200	log_id	Individual internet sessions

The project brief describes these same six connected tables and states that the dataset contains 20,000 records, with

6. EER Diagram

The following diagram is a faithful report reconstruction of the supplied MySQL Workbench EER model. Table names, columns, and foreign-key paths are derived from the supplied .mwb model and SQL source.



GroundStations is an independent reference table in the supplied relational schema; its country field supports geography-based analysis.

Relationship summary: Customers 1→many Subscriptions; Satellites 1→many Subscriptions; Satellites 1→many MaintenanceRecords; Subscriptions 1→many UsageLogs. GroundStations is a standalone table in the relational schema; its country field supports the geography-based scenario.

The source brief specifically calls out these relationship directions and asks that the supplied EER diagram be treated as

7. Database Schema & Data Dictionary

Satellites

Column	Data type	Key / rule	Constraint / meaning
satellite_id	INT	PK	PRIMARY KEY
satellite_name	VARCHAR(30)	UNIQUE, NOT NULL	Unique satellite identifier/name
orbit_altitude_km	INT	CHECK > 0	Orbit altitude in km
launch_date	DATE	NOT NULL	Launch date
status	VARCHAR(20)	DEFAULT 'Active'	Operational status

GroundStations

Column	Data type	Key / rule	Constraint / meaning
station_id	INT	PK	PRIMARY KEY
station_name	VARCHAR(50)	UNIQUE, NOT NULL	Ground-station name
country	VARCHAR(50)	NOT NULL	Station country
capacity_mbps	INT	CHECK > 0	Station capacity in Mbps

Customers

Column	Data type	Key / rule	Constraint / meaning
customer_id	INT	PK	PRIMARY KEY
customer_name	VARCHAR(60)	NOT NULL	Customer name
country	VARCHAR(50)	NOT NULL	Customer country
plan_type	VARCHAR(20)	DEFAULT 'Standard'	Customer plan category
signup_date	DATE	NOT NULL	Signup date

Subscriptions

Column	Data type	Key / rule	Constraint / meaning
subscription_id	INT	PK	PRIMARY KEY
customer_id	INT	FK	REFERENCES Customers(customer_id)
satellite_id	INT	FK	REFERENCES Satellites(satellite_id)
monthly_fee	DECIMAL(8,2)	CHECK > 0	Monthly fee
start_date	DATE	NOT NULL	Subscription start
status	VARCHAR(20)	DEFAULT 'Active'	Subscription status

MaintenanceRecords

Column	Data type	Key / rule	Constraint / meaning
record_id	INT	PK	PRIMARY KEY
satellite_id	INT	FK	REFERENCES Satellites(satellite_id)
maintenance_date	DATE	NOT NULL	Maintenance date
issue_type	VARCHAR(50)	—	Issue classification
resolved	VARCHAR(3)	DEFAULT 'No'; CHECK Yes/No	Resolution status
downtime_minutes	INT	CHECK >= 0	Downtime duration

UsageLogs

Column	Data type	Key / rule	Constraint / meaning
log_id	INT	PK	PRIMARY KEY
subscription_id	INT	FK	REFERENCES Subscriptions(subscription_id)
session_date	DATE	NOT NULL	Session date
data_used_gb	DECIMAL(6,2)	CHECK >= 0	Data consumed in GB
session_duration_minutes	INT	CHECK > 0	Session duration
latency_ms	INT	CHECK >= 0	Latency in milliseconds

8. Data Integrity & Constraints

Constraint	Applied in project	Business value
PRIMARY KEY	Every table	Prevents duplicate row identifiers and provides stable row identity.

Constraint	Applied in project	Business value
FOREIGN KEY	Subscriptions, MaintenanceRecords, UsageLogs	Protects referential integrity between operational entities.
UNIQUE	Satellite / station names	Prevents duplicate names where the model requires uniqueness.
NOT NULL	Required identifiers/dates and selected attributes	Prevents incomplete records in required fields.
DEFAULT	Statuses and selected categorical fields	Provides predictable values when an explicit value is omitted.
CHECK	Altitude, capacity, fees, resolution, downtime, usage, duration, latency	Rejects values outside the project's allowed domain.

The initial Satellites definition demonstrates the pattern clearly: `satellite_id` is a primary key, `satellite_name` is unique and required, `orbit_altitude_km` must be positive, `launch_date` is required, and `status` defaults to Active.

9. SQL Concepts Demonstrated

Category	Verified techniques in source
Basic SQL	SELECT, WHERE, ORDER BY, DISTINCT, LIMIT
Aggregation	COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING
Joins	INNER JOIN and LEFT JOIN
Subqueries / CTEs	Multi-step calculations and filtered intermediate results
Conditional logic	CASE classifications for quality, usage, generation, utilization
Window functions	LAG, LEAD, ROW_NUMBER, RANK, DENSE_RANK, NTILE, running counts / moving averages
Date & time	YEAR, MONTH, QUARTER, DATEDIFF, DATE_SUB, CURDATE, TIME, DAYNAME
String functions	CONCAT, TRIM, UPPER, LOWER, LEFT, SUBSTRING, LENGTH
DML / DDL	INSERT, UPDATE, DELETE, ALTER TABLE
Optimization	CREATE INDEX, composite index, SHOW INDEX
Database objects	CREATE VIEW
Security	CREATE ROLE, GRANT, REVOKE, FLUSH PRIVILEGES

RIGHT JOIN, FULL OUTER JOIN simulation, and stored procedures/functions are not demonstrated by the supplied SQL. CREATE USER is also not present; the security implementation uses roles and grants/revokes instead. This distinction is intentional to keep the report source-accurate.

10. 75 Real-World SQL Business Scenarios

The project contains 75 scenarios designed to simulate operational questions in a satellite internet company. The scenario list below preserves the supplied requirements while grouping them into practical categories.

Customer & Subscription Analytics

No.	Scenario
1	Work this out in two stages: first tally up each subscription's total number of sessions this month, then use that to shortlist subscriptions with fewer than 5 — these may be at risk of cancellation.
2	A satellite previously marked as retired has just been reactivated after a successful repair maneuver. Update its status to reflect this.

No.	Scenario
3	For each subscription, looking at its sessions in date order, show what the latency was on the reading immediately before the current one — this helps catch any sudden, unexplained connection quality drop.
4	Calculate a combined usage score per customer using their total data consumption and session count as an intermediate result, then use that to rank countries by overall network engagement.
5	Identify any subscription that has never logged a single usage session — a red flag for the customer success team to investigate.
8	A billing system recalibration means every monthly fee under one specific plan type needs to be adjusted by a fixed percentage. Apply this change to just that plan type's subscriptions.
9	For each plan type, build a running count of new subscriptions signed up over time, ordered by start date, to visualize how each plan has grown.
11	Within each country, order customers by their total data usage, highest first, to identify the network's heaviest users market by market.
13	Prepare a network report listing every subscription along with the customer's name and the satellite it's connected to, all on a single line.
15	Group all usage logs by subscription, and flag subscriptions with fewer than 3 total sessions recorded this month — these customers may be about to churn.
17	For the customer support dashboard, combine each customer's name and country into a single readable label, separated by a comma.
23	Count how many active subscriptions currently exist under each plan type.
24	Split all customers into three roughly equal usage tiers based on total data consumed, to help design three different targeted upsell campaigns.
26	First tally each country's total customer count, then bring in revenue details to see which countries are generating the most revenue per customer.
28	A data-entry mistake caused an entire batch of customers to be logged under the wrong country. Correct this in one sweep rather than customer by customer.
40	For each subscription, show what the very next session's data usage will be, placed right alongside the current one, to help spot a pattern building up across consecutive sessions.
46	Some customer names were typed with inconsistent capitalization during onboarding. Produce a clean, uniformly capitalized list for the billing system.
47	For each customer, work out how many days have passed since they signed up — this determines who's eligible for the one-year loyalty discount.
48	List every subscription along with its most recent usage session, making sure subscriptions that haven't logged any usage yet still appear on the list rather than disappearing.
60	Break this into two steps: first find each subscription's most recent session date, then use that to identify which subscriptions have gone quiet in the last 30 days.
62	The finance team wants to know, in one number, the total monthly revenue currently being generated from all active subscriptions.
63	A new customer support agent is joining the team and should only be able to look at customer and subscription data, never change any of it. Set up their access accordingly.
65	A support agent tries to save a new subscription without linking it to a customer. What should prevent that incomplete record from being saved?
70	Identify every subscription belonging to whichever single satellite currently has the highest number of subscribers.
72	For a specific customer flagged for unusual activity, pull all their sessions that occurred between midnight and 5 AM.
75	The customer support team needs a simplified, always-ready snapshot showing each customer's name, plan type, assigned satellite, and most recent session, without agents having to reconstruct the full picture from scratch on every call.

Satellite Operations

No.	Scenario
2	A satellite previously marked as retired has just been reactivated after a successful repair maneuver. Update its status to reflect this.
10	List all satellites that were launched in the same year as the satellite with the most subscribers.
19	Some satellites may not be serving a single active subscription right now. Identify exactly which ones, since this is unused capacity worth flagging.
22	Flag each satellite as 'Overloaded', 'Balanced', or 'Underutilized' based on how many subscriptions are currently linked to it relative to its typical capacity.
38	For the infrastructure report, work out the average orbit altitude across the entire satellite fleet.
43	Cross-check satellites against maintenance activity to identify any satellite that has never had a single maintenance record — either remarkably reliable or worryingly unmonitored.
45	An engineer accidentally types a satellite's orbit altitude as a negative number. How do you make sure this kind of entry is rejected the moment it's submitted?
50	Sort each satellite's launch date into 'Legacy' (before 2021), 'Mid-Generation' (2021-2023), or 'New Generation' (after 2023), to help engineers prioritize fleet upgrades.
59	Identify every satellite that was launched more than five years ago — these units are approaching the end of their expected operational life.
64	Going forward, the network wants to record which manufacturer built each satellite. Adjust the satellite records so this detail can be captured.
66	Count the subscriptions currently linked to each satellite, and shortlist only the satellites carrying more than 20 subscribers — these may be nearing capacity.
67	Two onboarding agents in different regions accidentally try to register the same satellite name twice during a fleet update. What should stop this from happening?
68	For every satellite, surface the single highest-usage subscription linked to it without losing any of that satellite's other subscription rows in the process — every subscription should still show up in the result.

Network Performance

No.	Scenario
3	For each subscription, looking at its sessions in date order, show what the latency was on the reading immediately before the current one — this helps catch any sudden, unexplained connection quality drop.
18	The engineering dashboard constantly filters by satellite and maintenance date together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?
20	The quarterly network health report needs a single consolidated result: for every satellite, the number of subscriptions it serves, their average session latency, the total downtime from maintenance, and a clear flag for any satellite whose average latency has crossed an unacceptable threshold.
25	Based on session latency, tag each usage log as 'Excellent', 'Acceptable', or 'Poor' connection quality, to help support agents quickly diagnose complaints.
41	For each satellite, smooth out momentary reporting glitches by calculating an average latency using only the current reading and the two immediately before it.
42	Find the lowest latency ever recorded across the network — this benchmark could be used in the next marketing campaign.
52	The system currently lets someone log a session latency of -40 milliseconds by mistake. What change would permanently make this impossible going forward?
54	Find every plan type that has at least one subscriber in a country where the average latency is above 90 milliseconds.

No.	Scenario
55	Build an intermediate satellite-by-satellite average latency figure, then use it to flag every satellite performing worse than the network-wide average.
56	Track down which satellite is linked to the subscription that recorded the single highest data usage anywhere in the entire dataset.
69	Identify which countries have an average session data usage above 20 GB — these markets may need additional network capacity.

Usage Analytics

No.	Scenario
4	Calculate a combined usage score per customer using their total data consumption and session count as an intermediate result, then use that to rank countries by overall network engagement.
31	The scheduling team wants to know which day of the week sees the highest data usage, to plan network maintenance windows around quieter periods.
34	Pull every usage session recorded during the month of March this year, to study seasonal demand patterns.
35	Label each usage session as 'Light' (under 5 GB), 'Moderate' (5-20 GB), or 'Heavy' (above 20 GB), so the network team can understand consumption patterns at a glance.
40	For each subscription, show what the very next session's data usage will be, placed right alongside the current one, to help spot a pattern building up across consecutive sessions.
42	Find the lowest latency ever recorded across the network — this benchmark could be used in the next marketing campaign.
49	Across the entire network, order every usage session by data consumed to surface the season's heaviest sessions, making sure that if two sessions used the exact same amount of data, neither one gets unfairly skipped in the ordering.
51	Find the single highest data usage recorded in one session — this could be a business customer running a large upload job worth understanding better.
53	Pull every usage log belonging specifically to the one subscription that recorded the highest singlesession data usage in the system.
57	The regional expansion team wants to flag any ground station name that's unusually long (more than 30 characters) so it fits the map interface properly.
58	Work out how many distinct countries the customer base currently spans.
74	For each subscription, flag any of its own sessions where the data usage is noticeably higher than that same subscription's own typical usage — a possible sign of a compromised account or a new use case.

Revenue & Finance

No.	Scenario
8	A billing system recalibration means every monthly fee under one specific plan type needs to be adjusted by a fixed percentage. Apply this change to just that plan type's subscriptions.
14	For each plan type, calculate the total monthly revenue it generates, and highlight any plan type contributing less than 10% of total revenue — a pricing strategy question worth raising.
23	Count how many active subscriptions currently exist under each plan type.
26	First tally each country's total customer count, then bring in revenue details to see which countries are generating the most revenue per customer.
61	The billing supervisor needs the ability to update subscription fee records directly, unlike the rest of the support staff. Set up an access level that reflects this difference.
62	The finance team wants to know, in one number, the total monthly revenue currently being generated from all active subscriptions.

No.	Scenario
71	The finance dashboard wants something it can plug straight into its reporting tool that always reflects each plan type's current total monthly revenue, without re-deriving it manually every cycle.

Maintenance & Reliability

No.	Scenario
16	For a satellite currently under review, work out how many days passed between its launch date and its very first maintenance record.
18	The engineering dashboard constantly filters by satellite and maintenance date together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?
20	The quarterly network health report needs a single consolidated result: for every satellite, the number of subscriptions it serves, their average session latency, the total downtime from maintenance, and a clear flag for any satellite whose average latency has crossed an unacceptable threshold.
21	Group maintenance records by satellite and identify which satellites have had more than 10 recorded incidents — these may need to be prioritized for early retirement.
36	Calculate the average downtime per maintenance event, to understand how disruptive a typical incident really is.
39	The network engineering team keeps asking for the same list — every satellite with its total count and total downtime. Set up something they can pull directly, without you explaining the underlying tables every time they ask.
43	Cross-check satellites against maintenance activity to identify any satellite that has never had a single maintenance record — either remarkably reliable or worryingly unmonitored.

Data Quality

No.	Scenario
6	After a recent billing system cleanup, a few usage logs may now point to a subscription ID that no longer exists. Track down these orphaned records.
28	A data-entry mistake caused an entire batch of customers to be logged under the wrong country. Correct this in one sweep rather than customer by customer.
30	A usage session arrives logged against a subscription ID that was never created in the system. Explain what should happen to this record and why it must not slip through quietly.
32	A bulk import left several ground station names with accidental leading or trailing spaces. Clean these up before they go into the network status report.
33	A batch of very old usage logs from a discontinued legacy billing cycle is cluttering the database and is no longer needed for any analysis. Remove exactly those records.
45	An engineer accidentally types a satellite's orbit altitude as a negative number. How do you make sure this kind of entry is rejected the moment it's submitted?
46	Some customer names were typed with inconsistent capitalization during onboarding. Produce a clean, uniformly capitalized list for the billing system.
52	The system currently lets someone log a session latency of -40 milliseconds by mistake. What change would permanently make this impossible going forward?
65	A support agent tries to save a new subscription without linking it to a customer. What should prevent that incomplete record from being saved?
67	Two onboarding agents in different regions accidentally try to register the same satellite name twice during a fleet update. What should stop this from happening?

Performance Optimization

No.	Scenario
7	Support agents constantly search usage logs using a specific subscription's ID to pull up its full session history instantly, but this lookup has started to feel sluggish now that the log table has grown into the tens of thousands. What would you do about it?
18	The engineering dashboard constantly filters by satellite and maintenance date together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?
29	After several rounds of database changes, nobody quite remembers what was set up to speed up lookups on the usage logs table. Check what's currently in place. show index from usagelogs;

Security & Access Control

No.	Scenario
37	A contractor's engagement with the network operations team has just ended, and their access to the database needs to be fully withdrawn with immediate effect.
61	The billing supervisor needs the ability to update subscription fee records directly, unlike the rest of the support staff. Set up an access level that reflects this difference.
63	A new customer support agent is joining the team and should only be able to look at customer and subscription data, never change any of it. Set up their access accordingly.

Database Evolution & Reporting

No.	Scenario
39	The network engineering team keeps asking for the same list — every satellite with its total count and total downtime. Set up something they can pull directly, without you explaining the underlying tables every time they ask.
64	Going forward, the network wants to record which manufacturer built each satellite. Adjust the satellite records so this detail can be captured.
65	A support agent tries to save a new subscription without linking it to a customer. What should prevent that incomplete record from being saved?
67	Two onboarding agents in different regions accidentally try to register the same satellite name twice during a fleet update. What should stop this from happening?
71	The finance dashboard wants something it can plug straight into its reporting tool that always reflects each plan type's current total monthly revenue, without re-deriving it manually every cycle.
75	The customer support team needs a simplified, always-ready snapshot showing each customer's name, plan type, assigned satellite, and most recent session, without agents having to reconstruct the full picture from scratch on every call.

The project brief explicitly identifies indexing, revenue, maintenance, satellite capacity, data quality, access control, views,

11. Selected SQL Query Showcase

Fourteen queries were selected because they collectively demonstrate relational joins, aggregation, CTEs, subqueries, CASE logic, window functions, indexing, constraints, and reusable views. The code below is reproduced from the supplied SQL source rather than rewritten.

Scenario 3 — For each subscription, looking at its sessions in date order, show what the latency was on the reading immediately before the current one — this helps catch any sudden, unexplained connection quality drop.

SQL concept: LAG window function

```
select subscription_id,session_date,latency_ms,
lag(latency_ms) over (partition by subscription_id order by session_date) as Previous_latency
from usagelogs;
```

LAG partitions the usage stream by subscription and exposes the immediately preceding latency reading.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 4 — Calculate a combined usage score per customer using their total data consumption and session count as an intermediate result, then use that to rank countries by overall network engagement.

SQL concept: CTE + aggregation + RANK

```
With CustomerUsage as
(select c.customer_id,c.country,
sum(u.data_used_gb) as total_data,
count(u.log_id) as session_count,
(sum(u.data_used_gb) +count(u.log_id)) as Usage_score
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
inner join usagelogs u
on s.subscription_id = u.subscription_id
group by c.customer_id,c.country)
select country,
sum(Usage_score) as Total_engagement,
sum(total_data) as Country_total_data,
sum(session_count) as Country_total_sessions,
rank() over (order by sum(Usage_score) desc) as Engagement_rank
from CustomerUsage
group by country
order by Total_engagement desc;
```

The CTE calculates customer-level data, session count, and a combined score before aggregating to country and ranking engagement.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 7 — Support agents constantly search usage logs using a specific subscription's ID to pull up its full session history instantly, but this lookup has started to feel sluggish now that the log table has grown into the tens of thousands. What would you do about it?

SQL concept: Indexing

```
create index idx_usage_subscription
on usagelogs(subscription_id);
```

An index on subscription_id supports the common usage-log lookup path.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 18 — The engineering dashboard constantly filters by satellite and maintenance date together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?

SQL concept: Composite index

```
create index idx_satellite_maintenancedate
on maintenancerecords(satellite_id, maintenance_date);
show index from maintenancerecords;
```

A composite index on satellite_id and maintenance_date matches the two-column filter pattern described by the scenario.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 22 — Flag each satellite as 'Overloaded', 'Balanced', or 'Underutilized' based on how many subscriptions are currently linked to it relative to its typical capacity.

SQL concept: CTE + CASE

```
with Satellite_usage as
(select sa.satellite_id,sa.satellite_name,
count(s.subscription_id) as Subscription_count
from satellites sa
left join subscriptions s
on sa.satellite_id = s.satellite_id
group by sa.satellite_id,sa.satellite_name)
select satellite_id,satellite_name,subscription_count,
case
when subscription_count > (select avg(subscription_count)*1.20
from Satellite_usage) then 'Overloaded'
when subscription_count < (select avg(subscription_count)*0.80
```

```

from satellite_usage) then 'Underutilized'
else 'Balanced'
end as Utilization_status
from Satellite_usage;

```

The CTE first calculates satellite subscription counts; CASE then classifies utilization relative to the average count.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 24 — Split all customers into three roughly equal usage tiers based on total data consumed, to help design three different targeted upsell campaigns.

SQL concept: CTE + NTILE

```

with CustomerUsage as
(select c.customer_id, c.customer_name,
sum(u.data_used_gb) as Total_data
from customers c
left join subscriptions s
on c.customer_id = s.customer_id
left join usagelogs u
on s.subscription_id = u.subscription_id
group by c.customer_id, c.customer_name)
select customer_id, customer_name, total_data,
ntile(3) over (order by total_data) as Usage_tier
from CustomerUsage;

```

NTILE(3) splits customers into three ordered usage groups after customer-level total data is calculated.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 25 — Based on session latency, tag each usage log as 'Excellent', 'Acceptable', or 'Poor' connection quality, to help support agents quickly diagnose complaints.

SQL concept: CASE classification

```

select log_id, subscription_id, session_date, latency_ms,
case
when latency_ms < 100 then 'Excellent'
when latency_ms < 300 then 'Acceptable'
else 'Poor'
end as Connection_quality
from Usagelogs;

```

CASE converts numeric latency into the project's three connection-quality labels.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 39 — The network engineering team keeps asking for the same list — every satellite with its total count and total downtime. Set up something they can pull directly, without you explaining the underlying tables every time they ask.

SQL concept: View + JOIN + aggregation

```

create view SatelliteSummary as
select sa.satellite_id, sa.satellite_name,
count(s.subscription_id) as total_subscriber,
sum(downtime_minutes) as total_downtime
from Satellites sa
inner join Subscriptions s
on sa.satellite_id = s.satellite_id
inner join MaintenanceRecords m
on s.satellite_id = m.satellite_id
group by sa.satellite_id, sa.satellite_name
order by sa.satellite_id;
select * from SatelliteSummary;

```

A view packages recurring satellite subscriber and downtime reporting into a reusable object.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 40 — For each subscription, show what the very next session's data usage will be, placed right alongside the current one, to help spot a pattern building up across consecutive sessions.

SQL concept: LEAD window function

```
select subscription_id, session_date, data_used_gb,
lead(data_used_gb) over (partition by subscription_id order by session_date, log_id) as Next_session_data_usage
from UsageLogs;
```

LEAD exposes the next session's data usage within each subscription's chronological sequence.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 45 — An engineer accidentally types a satellite's orbit altitude as a negative number. How do you make sure this kind of entry is rejected the moment it's submitted?

SQL concept: ALTER TABLE + CHECK constraint

```
alter table satellites
add constraint check_orbit_altitude
check (orbit_altitude_km >= 0);

insert into satellites
(satellite_id, satellite_name, orbit_altitude_km, launch_date)
values
(40, 'OrbitLink-040', -638, '2024-12-14', 'Retired');
```

ALTER TABLE adds a CHECK constraint intended to prevent negative orbit altitude values.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 54 — Find every plan type that has at least one subscriber in a country where the average latency is above 90 milliseconds.

SQL concept: CTE + DISTINCT

```
with CountryLatency as
(select c.country,
avg(u.latency_ms) as avg_latency
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
inner join usagelogs u
on s.subscription_id = u.subscription_id
group by c.country)
select distinct
c.plan_type
from customers c
inner join CountryLatency cl
on c.country = cl.country
where cl.avg_latency > 90;
```

A country-level latency CTE is joined back to customers to return distinct plan types exposed to high-latency markets.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 55 — Build an intermediate satellite-by-satellite average latency figure, then use it to flag every satellite performing worse than the network-wide average.

SQL concept: Subquery + CASE + aggregation

```
select s.satellite_id,
avg(u.latency_ms) as Satellite_avg_latency,
(select avg(latency_ms) from usagelogs) as Network_avg_latency,
case
when avg(u.latency_ms) > (select avg(latency_ms) from usagelogs)
then 'Worse than network average'
else 'Better than network average'
end as Performance_score
from usagelogs u
inner join subscriptions s
on u.subscription_id = s.subscription_id
group by s.satellite_id;
```

Satellite averages are compared with the network-wide average using a scalar subquery and CASE.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 71 — The finance dashboard wants something it can plug straight into its reporting tool that always reflects each plan type's current total monthly revenue, without re-deriving it manually every cycle.

SQL concept: View + JOIN + GROUP BY

```
create view PlanTypeRevenue as
select plan_type,
sum(monthly_fee) as Total_monthly_revenue
from subscriptions s
inner join customers c
on s.customer_id = c.customer_id
group by plan_type;
select * from PlanTypeRevenue;
```

The view provides current plan-type revenue without requiring the reporting consumer to rebuild the join and aggregation.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

Scenario 75 — The customer support team needs a simplified, always-ready snapshot showing each customer's name, plan type, assigned satellite, and most recent session, without agents having to reconstruct the full picture from scratch on every call.

SQL concept: View + CTE + ROW_NUMBER + LEFT JOIN

```
create view CustomerSupportSnapshot as
with LatestUsage as
(select u.subscription_id,u.session_date,
row_number() over (partition by u.subscription_id order by u.session_date desc, u.log_id desc) as rnk
from UsageLogs u)
select c.customer_name,c.plan_type,sa.satellite_name,lu.session_date AS Most_recent_session
from Customers c
inner join Subscriptions s
on c.customer_id = s.customer_id
left join Satellites sa
on s.satellite_id = sa.satellite_id
left join LatestUsage lu
on s.subscription_id = lu.subscription_id
and lu.rnk = 1;
select * from CustomerSupportSnapshot;
```

A CTE and ROW_NUMBER identify the latest session per subscription; LEFT JOIN preserves subscriptions with no usage.

Business value: turns raw operational records into a reusable engineering, support, finance, or monitoring signal.

12. Advanced SQL Analysis

Window functions

The source uses window functions for sequential and comparative analysis: LAG for previous latency, LEAD for next-session usage, ROW_NUMBER for latest-session selection, RANK for within-satellite top usage, DENSE_RANK for tied usage ordering, NTILE for three customer usage tiers, and a moving average over the current plus two preceding readings.

CTEs and multi-step analysis

Several scenarios first create an intermediate dataset and then perform a second calculation. This pattern is particularly visible in customer engagement, subscription recency, satellite utilization, country latency, and customer support snapshots.

Conditional logic

- Connection quality: Excellent / Acceptable / Poor
- Usage: Light / Moderate / Heavy
- Satellite generation: Legacy / Mid-Generation / New Generation
- Satellite utilization: Overloaded / Balanced / Underutilized
- Performance: Worse than network average / Better than network average

These classifications are explicitly part of the supplied scenarios and SQL implementation.

13. Data Quality & Validation

Quality issue	SQL approach
Prevent negative orbit altitude	CHECK constraint on Satellites.orbit_altitude_km
Prevent negative latency	CHECK constraint added to UsageLogs.latency_ms
Prevent duplicate satellite names	UNIQUE on Satellites.satellite_name
Prevent subscriptions without customers	NOT NULL modification for Subscriptions.customer_id
Detect orphaned usage logs	LEFT JOIN from UsageLogs to Subscriptions
Clean whitespace	TRIM on GroundStations.station_name
Standardize names	UPPER / LOWER / LEFT / SUBSTRING
Correct bad country batch	Single UPDATE by wrong country value

Detection vs prevention: the project demonstrates both. Detection queries locate orphaned, stale, unusual, or inconsistent records; DDL constraints prevent several classes of invalid data from being stored in the first place. The

14. Performance Optimization

Two indexes are explicitly created in the supplied SQL: `idx_usage_subscription` on `UsageLogs(subscription_id)`, and `idx_satellite_maintenancedate` on `MaintenanceRecords(satellite_id, maintenance_date)`. The first supports repeated subscription-history lookups; the second supports the combined satellite/date filter pattern.

```
CREATE INDEX idx_usage_subscription
ON usagelogs(subscription_id);

CREATE INDEX idx_satellite_maintenancedate
ON maintenancerecords(satellite_id, maintenance_date);
```

The project also includes `SHOW INDEX` statements to inspect the current indexing configuration. Indexes can reduce lookup work on frequently filtered columns, while adding indexes also introduces storage and write-maintenance overhead; therefore the best index is one aligned to actual query patterns.

15. Security & Access Control

Access level	Source implementation	Interpretation
billing_supervisor	CREATE ROLE + GRANT UPDATE	Allows subscription fee records to be updated; the source also demonstrates column-specific UPDATE on <code>monthly_fee</code> .
support_agent	CREATE ROLE + GRANT SELECT	The supplied SQL grants SELECT on Customers. It does not also grant SELECT on Subscriptions despite the scenario wording.
contractor_user	REVOKE ALL PRIVILEGES, GRANT OPTION	Removes privileges and grant capability from the named account.

No `CREATE USER` statement is present in the supplied SQL. The implemented security layer therefore demonstrates roles, grants, and revocation rather than full account provisioning.

16. KPI Summary

The following KPIs are calculated directly from the supplied INSERT dataset. They are descriptive findings from the provided records, not invented project assumptions.

20,000	40	25	700
Total records	Satellites	Ground stations	Customers

700 Subscriptions	25 Active satellites	662 Active subscriptions	18200 Usage sessions
335 Maintenance events	405,967.10 GB Total data consumed	68.84 ms Average latency	18 ms Lowest latency
82,893 min Maintenance downtime	247.44 min Avg downtime / event	752.75 km Avg orbit altitude	\$53,158.00 Active monthly revenue

Status mix: 25 Active, 9 Degraded, and 6 Retired satellites. Subscription status is 662 Active, 34 Suspended, and 4 Cancelled. Customer plan distribution is 421 Standard, 202 Premium, and 77 Business.

17. Key Business Insights

Customer base. 700 customers span 20 countries. Standard is the largest customer plan category (421 customers), followed by Premium (202) and Business (77).

Subscription activity. 662 of 700 subscriptions are Active (94.6%), while 34 are Suspended and 4 are Cancelled.

Revenue. The supplied subscription fees total \$56,096 across all subscriptions; active subscriptions contribute \$53,158 per month. Plan-type fee totals are Business \$19,173 (34.18%), Standard \$18,945 (33.77%), and Premium \$17,978 (32.05%).

Usage. 18,200 sessions consume 405,967.10 GB in total, averaging 22.31 GB per session. 54.8% of sessions are Heavy (>20 GB), 34.1% Moderate, and 11.1% Light.

Latency. Average latency is 68.84 ms. Under the project's CASE thresholds, 14,498 sessions are Excellent (<100 ms), 3,702 Acceptable (100–299 ms), and none Poor.

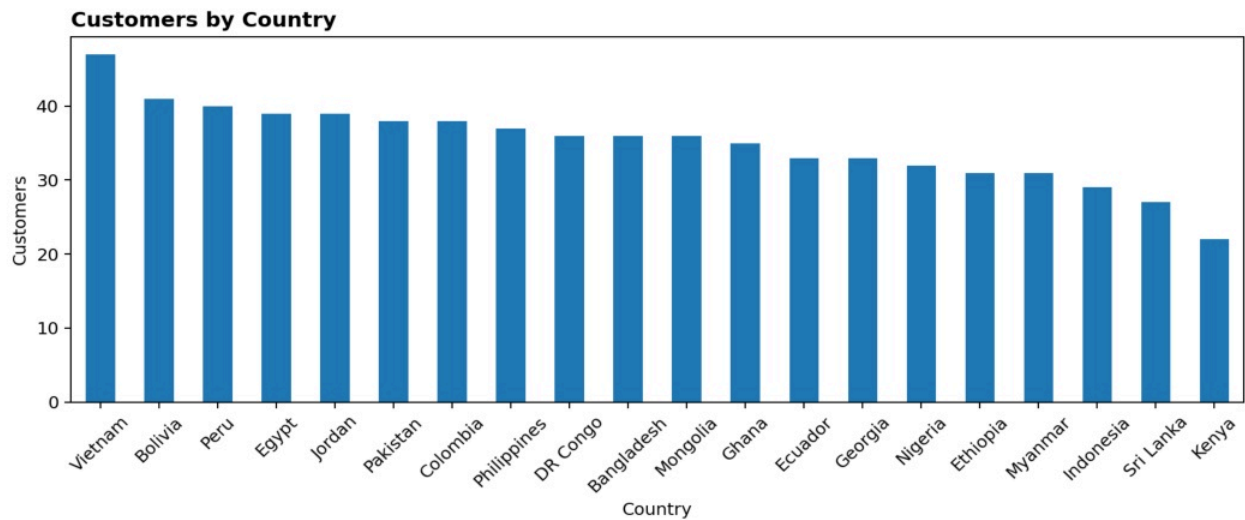
Satellite utilization. Satellite 3 has the highest number of active subscriptions at 27; the next highest are satellite 33 with 25 and satellite 28 with 24.

Maintenance. 335 maintenance events account for 82,893 minutes of downtime, averaging 247.44 minutes per event. Eleven satellites have more than 10 maintenance records.

Fleet age. 14 satellites were launched before 1 September 2021, making them more than five years old relative to the report date.

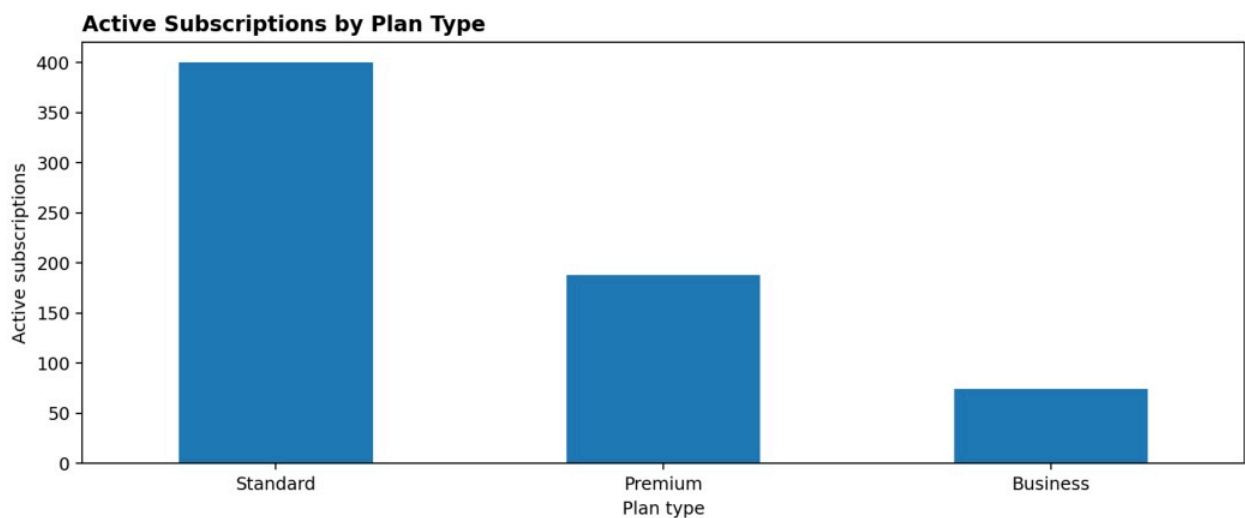
18. Visual Analytics

Customers by Country



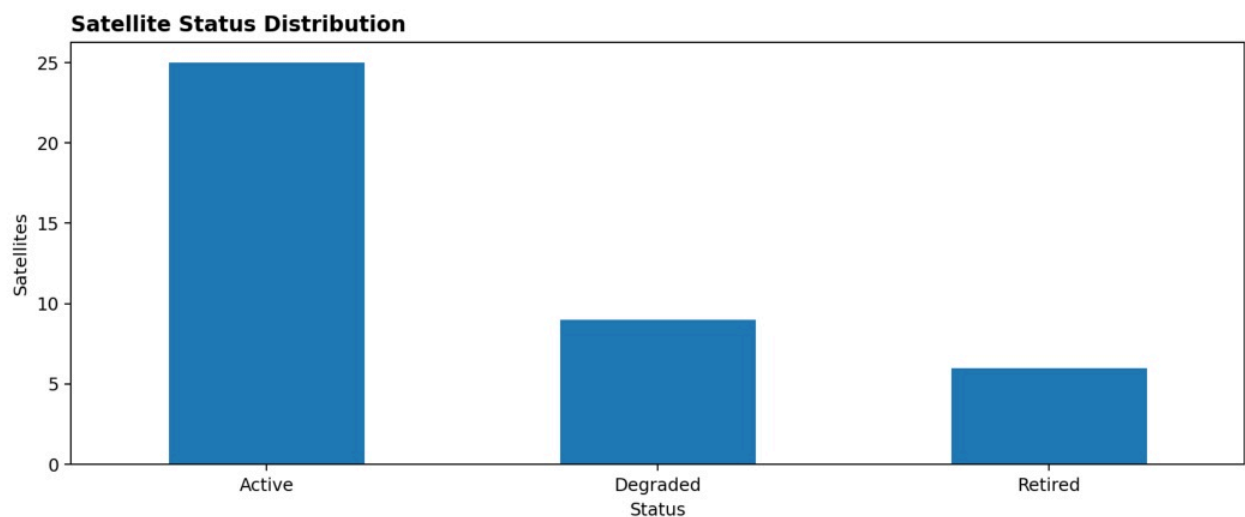
Interpretation: Customer concentration across the 20-country customer base. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Active Subscriptions by Plan Type



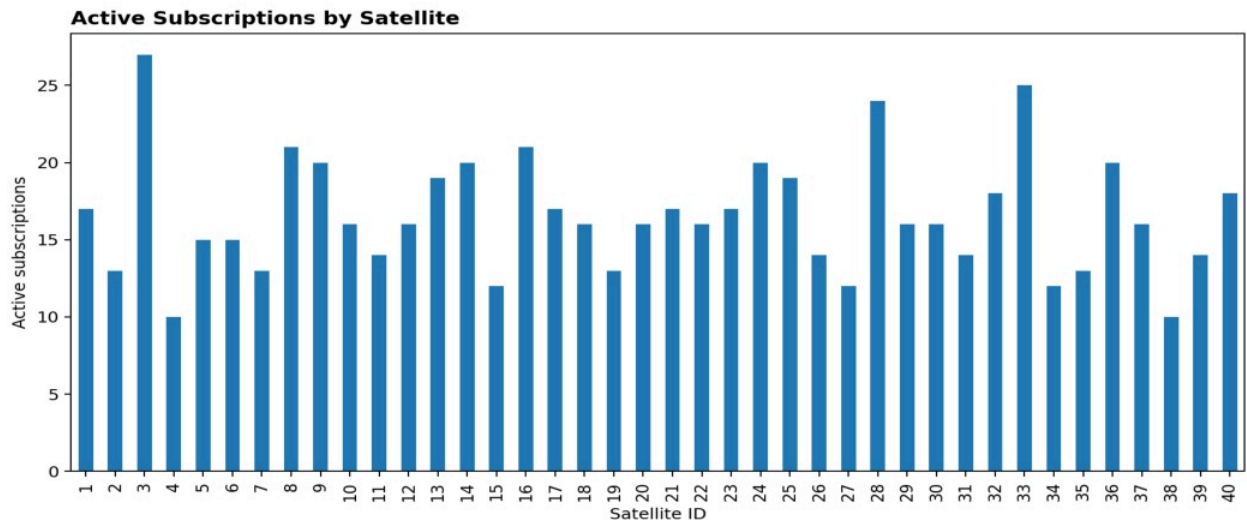
Interpretation: Current active subscription distribution by customer plan type. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Satellite Status Distribution



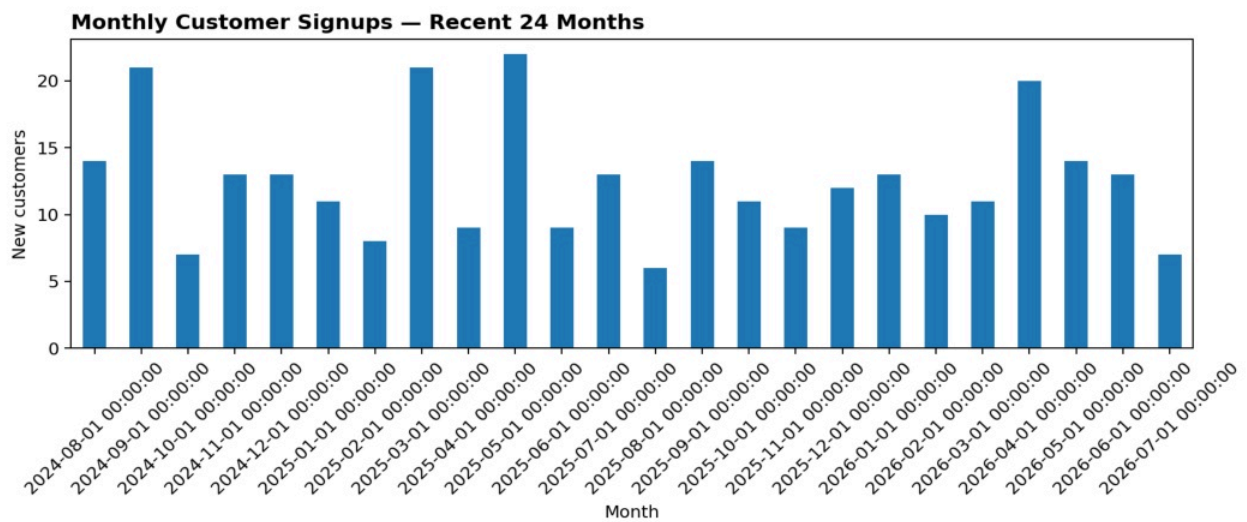
Interpretation: Fleet mix across Active, Degraded, and Retired states. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Active Subscriptions by Satellite



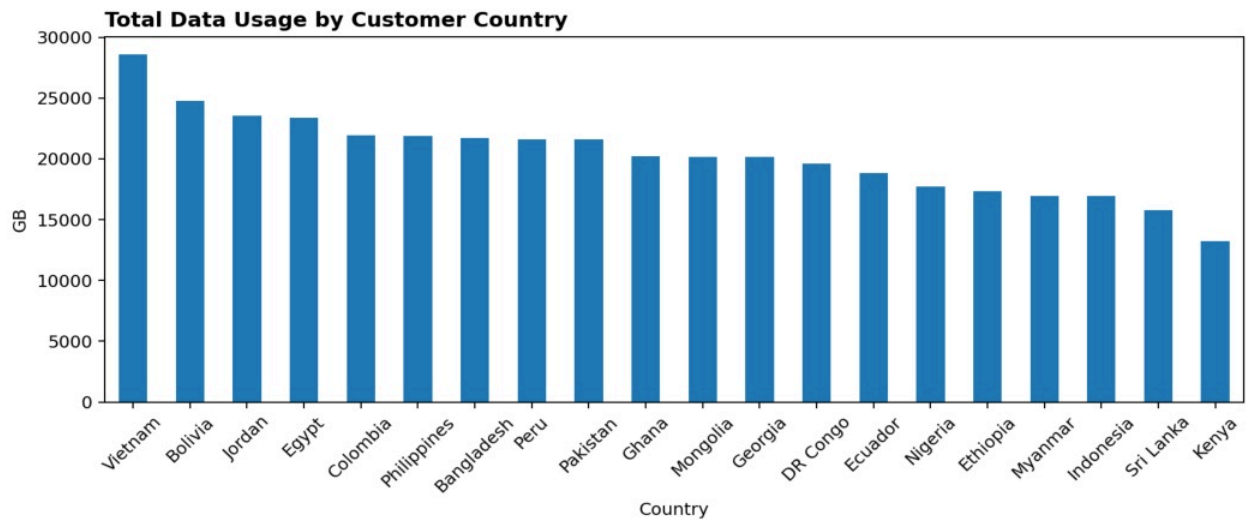
Interpretation: Subscriber concentration across the satellite fleet. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Monthly Customer Signups — Recent 24 Months



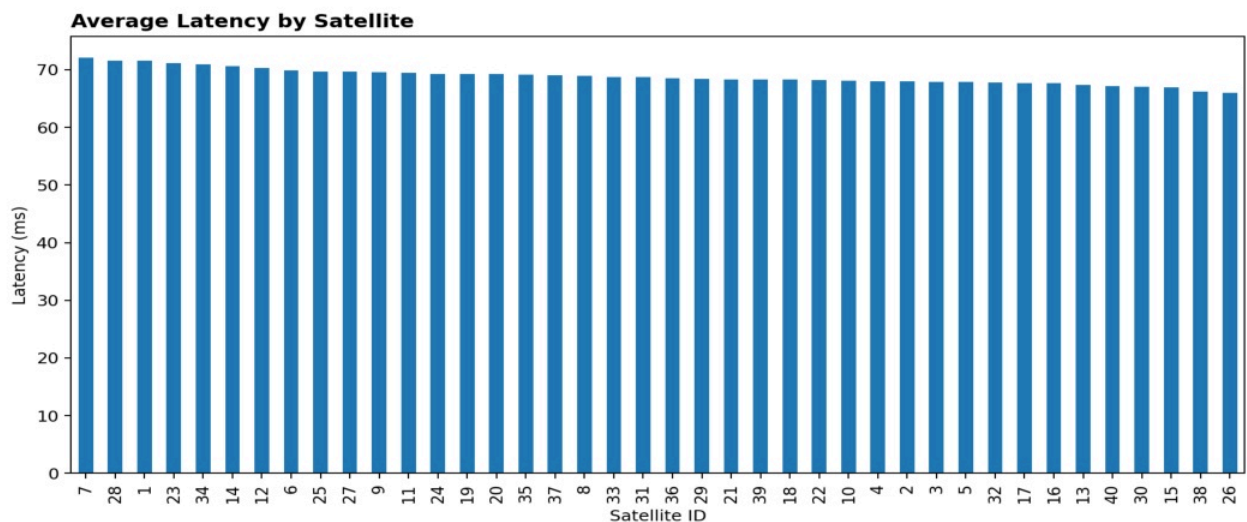
Interpretation: Recent signup cadence from the supplied customer dates. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Total Data Usage by Customer Country



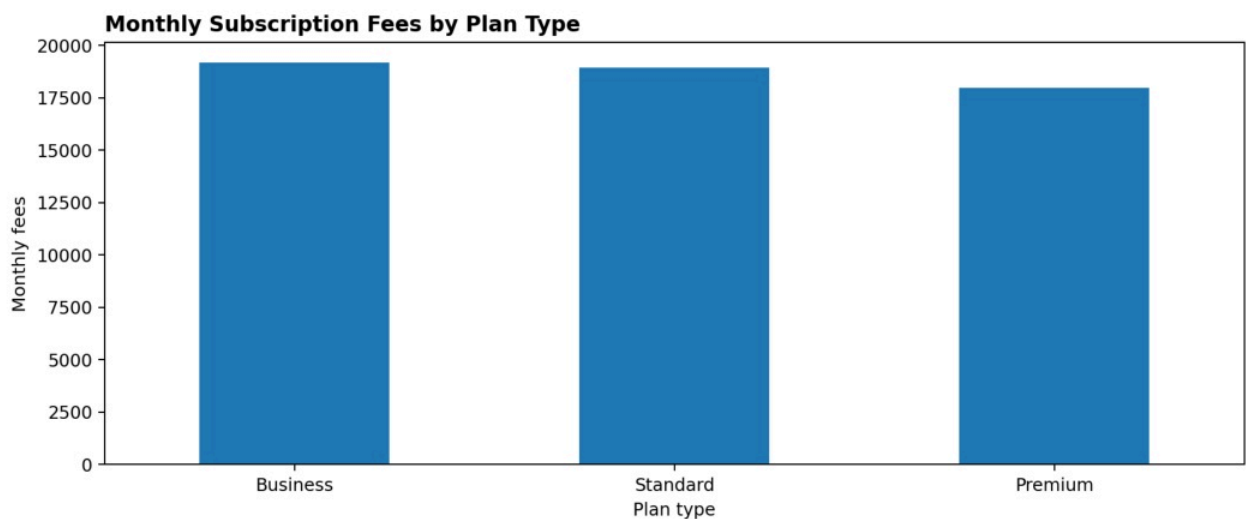
Interpretation: Total data consumption attributed to customer country. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Average Latency by Satellite



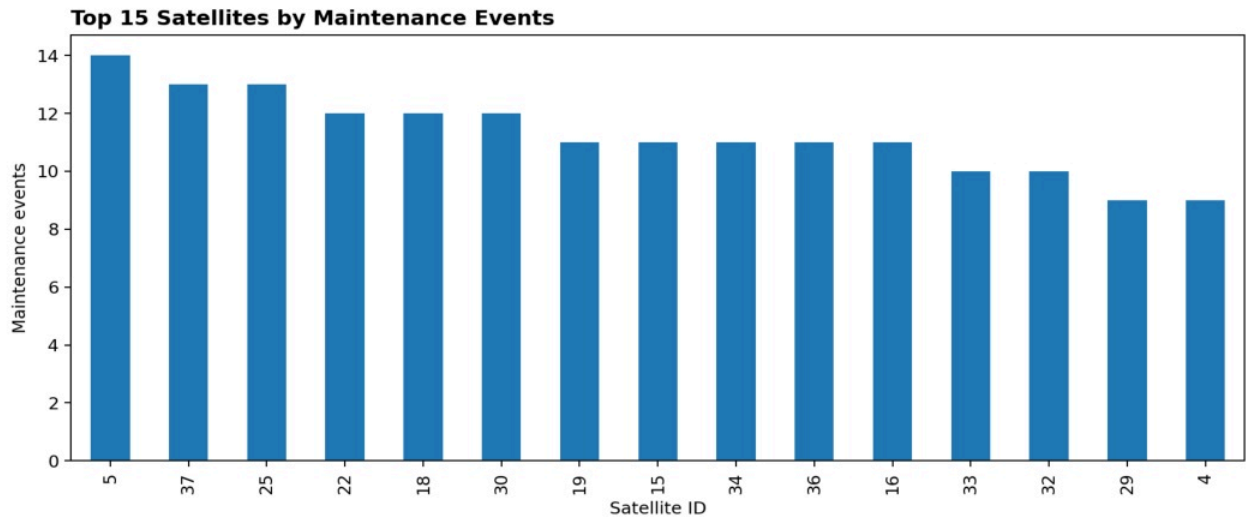
Interpretation: Average session latency associated with each satellite. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Monthly Subscription Fees by Plan Type



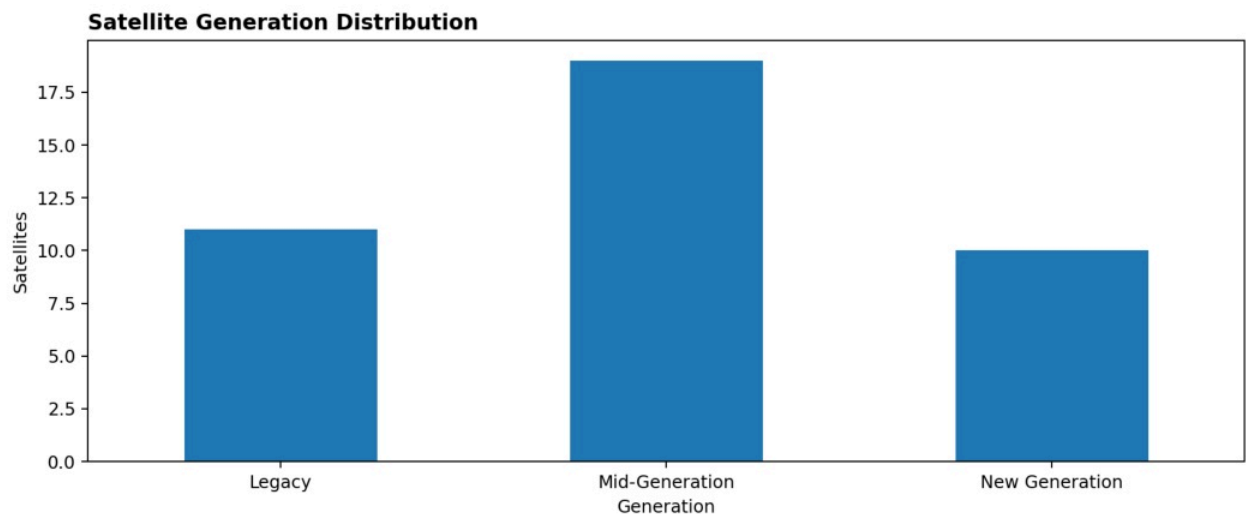
Interpretation: Monthly subscription fee contribution by plan. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Top 15 Satellites by Maintenance Events



Interpretation: Satellites with the most recorded maintenance events. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

Satellite Generation Distribution



Interpretation: Fleet classification using the scenario's launch-year tiers. **Method:** aggregated directly from the supplied INSERT dataset using the same business fields referenced by the scenario.

19. Operational Dashboard Concept

Dashboard area	Suggested content	SQL source
Network Overview	Total/active satellites, ground stations, customers, active subscriptions	Core counts and status filters
Satellite Health	Status, subscriber count, average latency, downtime	Satellite summary + latency aggregations
Customer Analytics	Country, plan, usage tiers, low-activity indicators	Customer/subscription/usage joins + NTILE
Financial Overview	Monthly revenue, plan revenue, revenue per customer	SUM + GROUP BY + PlanTypeRevenue view
Network Usage	Sessions, GB, heavy usage, peak periods	UsageLogs aggregations + CASE + date functions

The supplied views `SatelliteSummary`, `PlanTypeRevenue`, and `CustomerSupportSnapshot` are natural building blocks for reporting consumers. The scenario brief specifically asks for reusable views for recurring reporting.

20. SQL Project Workflow

01. Business Problem

02. Database Design

03. EER Modeling

04. Table Creation

05. Data Insertion

06. Data Validation

07. SQL Query Development

08. Advanced Analysis

09. Performance Optimization

10. Security

11. Business Insights

12. Reporting

This workflow connects the operational requirement to the final reporting layer: first define the relational model, then load and validate data, build increasingly sophisticated queries, optimize recurring access paths, apply security, and finally turn the results into business-facing insights.

21. Challenges & Solutions

Challenge	SQL solution	Business benefit
Large usage-log table	Indexing on <code>subscription_id</code>	Faster subscription-history lookups
Combined satellite/date filtering	Composite index	Better alignment with recurring filter pattern
Invalid numeric values	CHECK constraints	Stronger domain validation
Duplicate names	UNIQUE constraint	Prevents duplicate satellite/station names
Missing relationships	Foreign keys	Referential integrity
Complex analysis	CTEs / subqueries	Structured multi-step logic
Sequential telemetry	Window functions	Previous/next readings and moving averages
Repeated reporting	Views	Reusable reporting datasets
Different responsibilities	Roles / grants / revoke	Least-privilege-oriented access patterns

22. Learning Outcomes

- Relational database design and EER modeling
- DDL and DML
- Joins and aggregation
- Subqueries and CTEs
- Window functions and ranking

- Date/time and string manipulation
- Data cleaning and validation
- Indexing and query optimization
- Views and reusable reporting
- Database security and privileges
- Translating business requirements into SQL logic
- Communicating analytical results for engineering, finance, operations, and support

23. Future Enhancements

The following are recommendations only; they are not claimed to exist in the current project.

- Power BI / Tableau integration
- Automated network-health dashboard
- Predictive satellite maintenance
- Customer churn prediction
- Usage anomaly detection
- Automated high-latency alerts
- Capacity forecasting
- Satellite failure prediction
- Automated ETL pipelines
- Cloud database deployment
- Scheduled reporting

24. Conclusion

OrbitLink demonstrates practical SQL engineering by connecting a relational database model to realistic operational questions. The project is not limited to basic SELECT statements: it includes constraints, foreign keys, indexing, views, role-based privileges, multi-step analysis, and a broad set of window-function techniques.

The strongest portfolio value comes from the bridge between technical implementation and business purpose. Satellite health, customer engagement, usage behavior, revenue, maintenance reliability, data quality, and performance optimization are all expressed as SQL problems with an operational outcome. That makes the project suitable for demonstrating both SQL proficiency and analytical thinking in a portfolio or technical interview.

25. Appendix A — Complete Database Schema

```
CREATE TABLE Satellites (
  satellite_id INT PRIMARY KEY,
  satellite_name VARCHAR(30) NOT NULL UNIQUE,
  orbit_altitude_km INT CHECK (orbit_altitude_km > 0),
  launch_date DATE NOT NULL,
  status VARCHAR(20) DEFAULT 'Active'
);

CREATE TABLE GroundStations (
  station_id INT PRIMARY KEY,
  station_name VARCHAR(50) NOT NULL UNIQUE,
  country VARCHAR(50) NOT NULL,
  capacity_mbps INT CHECK (capacity_mbps > 0)
);

CREATE TABLE Customers (
```

```

customer_id INT PRIMARY KEY,
customer_name VARCHAR(60) NOT NULL,
country VARCHAR(50) NOT NULL,
plan_type VARCHAR(20) DEFAULT 'Standard',
signup_date DATE NOT NULL
);

CREATE TABLE Subscriptions (
subscription_id INT PRIMARY KEY,
customer_id INT,
satellite_id INT,
monthly_fee DECIMAL(8,2) CHECK (monthly_fee > 0),
start_date DATE NOT NULL,
status VARCHAR(20) DEFAULT 'Active',
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
FOREIGN KEY (satellite_id) REFERENCES Satellites(satellite_id)
);

CREATE TABLE MaintenanceRecords (
record_id INT PRIMARY KEY,
satellite_id INT,
maintenance_date DATE NOT NULL,
issue_type VARCHAR(50),
resolved VARCHAR(3) DEFAULT 'No' CHECK (resolved IN ('Yes','No')),
downtime_minutes INT CHECK (downtime_minutes >= 0),
FOREIGN KEY (satellite_id) REFERENCES Satellites(satellite_id)
);

CREATE TABLE UsageLogs (
log_id INT PRIMARY KEY,
subscription_id INT,
session_date DATE NOT NULL,
data_used_gb DECIMAL(6,2) CHECK (data_used_gb >= 0),
session_duration_minutes INT CHECK (session_duration_minutes > 0),
latency_ms INT CHECK (latency_ms >= 0),
FOREIGN KEY (subscription_id) REFERENCES Subscriptions(subscription_id)
);

```

26. Appendix B — Important SQL Objects

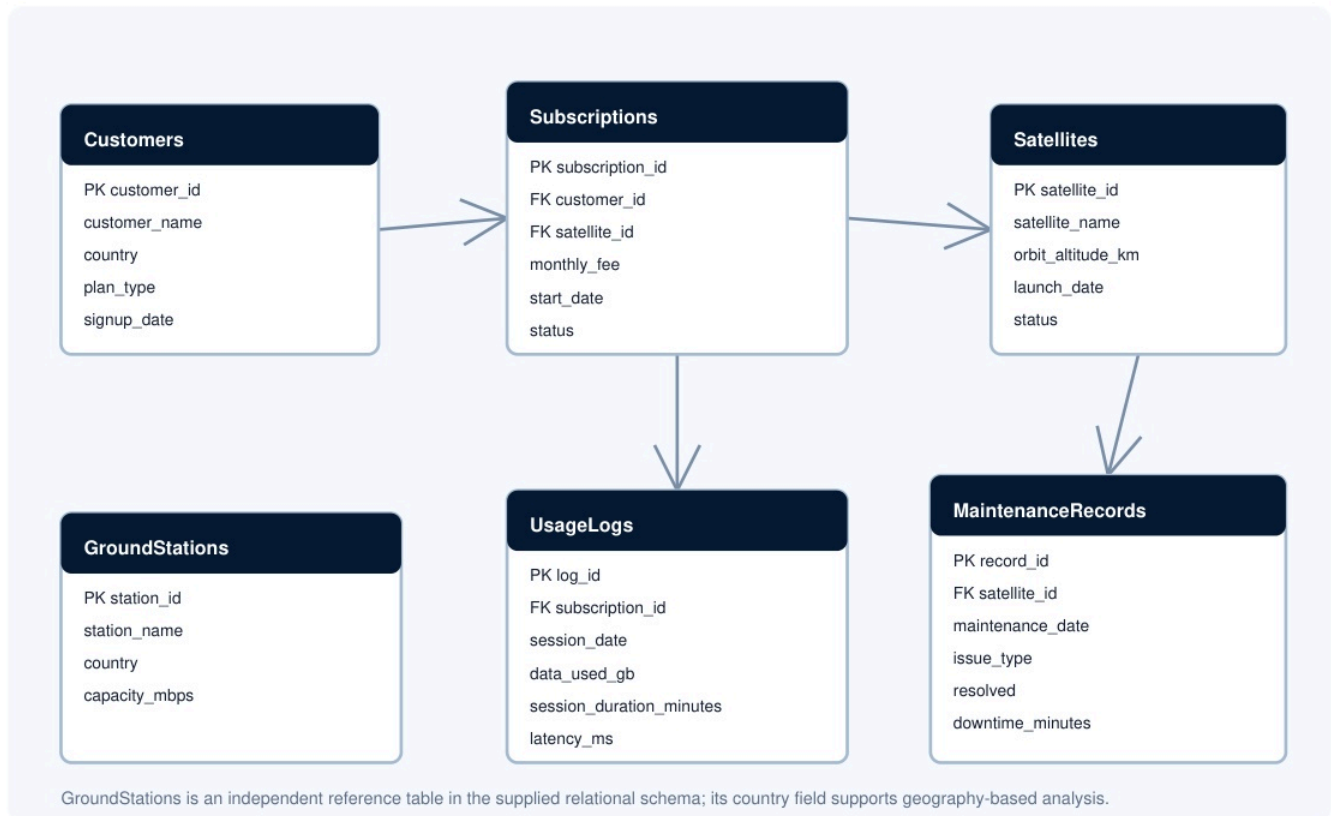
Type	Name	Purpose / Implementation
Index	idx_usage_subscription	UsageLogs(subscription_id)
Index	idx_satellite_maintenancedate	MaintenanceRecords(satellite_id, maintenance_date)
View	SatelliteSummary	Satellite subscriber count + maintenance downtime
View	PlanTypeRevenue	Plan-type monthly fee totals
View	CustomerSupportSnapshot	Customer, plan, satellite, and latest session
Role	billing_supervisor	UPDATE on Subscriptions; monthly_fee update demonstrated
Role	support_agent	SELECT on Customers demonstrated

27. Appendix C — Source QA & Implementation Notes

Area	Source observation	Report treatment
Scenario 2	Scenario says a retired satellite was reactivated; supplied SQL sets satellite_id 35 to 'Retired'.	Source implementation retained as-is; report does not silently correct it.
Scenario 20	Scenario requests a consolidated satellite health result; no SQL implementation follows the scenario in the supplied file.	Not explicitly implemented in the supplied project files.
Scenario 30	Scenario asks what should happen to an orphaned usage session; no SQL statement is supplied.	Requirement is explanatory rather than implemented as a query.
Scenario 45	Later ALTER TABLE adds CHECK >= 0 while the original Satellites column uses CHECK > 0.	Both source statements are documented; no reconciliation is imposed.

Area	Source observation	Report treatment
Scenario 63	Scenario asks for customer and subscription read-only access; supplied SQL grants SELECT on Customers only.	Subscription SELECT is not explicitly granted in the supplied implementation.
Scenario 73	Scenario describes ground-station/country matching; no SQL query follows in the supplied file.	Not explicitly implemented in the supplied project files.
Scenario 39	SatelliteSummary uses INNER JOINS to subscriptions and maintenance records.	As written, the view can omit satellites without matching rows in those joined tables.

28. Appendix D — EER Diagram



This diagram is reconstructed from the supplied MySQL Workbench model. The model contains six table figures—customers, groundstations, maintenancerecords, satellites, subscriptions, and usagelogs—and four foreign-key connection paths.

29. Appendix E — SQL Concepts Used

SQL concept	Status
SELECT	Demonstrated in supplied source
WHERE	Demonstrated in supplied source
ORDER BY	Demonstrated in supplied source
DISTINCT	Demonstrated in supplied source
LIMIT	Demonstrated in supplied source
COUNT	Demonstrated in supplied source
SUM	Demonstrated in supplied source
AVG	Demonstrated in supplied source

SQL concept	Status
MIN	Demonstrated in supplied source
MAX	Demonstrated in supplied source
GROUP BY	Demonstrated in supplied source
HAVING	Demonstrated in supplied source
INNER JOIN	Demonstrated in supplied source
LEFT JOIN	Demonstrated in supplied source
Subqueries	Demonstrated in supplied source
CTEs	Demonstrated in supplied source
CASE	Demonstrated in supplied source
LAG	Demonstrated in supplied source
LEAD	Demonstrated in supplied source
ROW_NUMBER	Demonstrated in supplied source
RANK	Demonstrated in supplied source
DENSE_RANK	Demonstrated in supplied source
NTILE	Demonstrated in supplied source
YEAR	Demonstrated in supplied source
MONTH	Demonstrated in supplied source
QUARTER	Demonstrated in supplied source
DATEDIFF	Demonstrated in supplied source
DATE_SUB	Demonstrated in supplied source
CURDATE	Demonstrated in supplied source
TIME	Demonstrated in supplied source
DAYNAME	Demonstrated in supplied source
CONCAT	Demonstrated in supplied source
TRIM	Demonstrated in supplied source
UPPER	Demonstrated in supplied source
LOWER	Demonstrated in supplied source
LEFT	Demonstrated in supplied source
SUBSTRING	Demonstrated in supplied source
LENGTH	Demonstrated in supplied source
INSERT	Demonstrated in supplied source
UPDATE	Demonstrated in supplied source
DELETE	Demonstrated in supplied source
ALTER TABLE	Demonstrated in supplied source
CREATE INDEX	Demonstrated in supplied source
SHOW INDEX	Demonstrated in supplied source

SQL concept	Status
CREATE VIEW	Demonstrated in supplied source
CREATE ROLE	Demonstrated in supplied source
GRANT	Demonstrated in supplied source
REVOKE	Demonstrated in supplied source
FLUSH PRIVILEGES	Demonstrated in supplied source

30. Appendix F — All 75 Business Scenarios

No.	Business requirement	Primary SQL concept
1	Work this out in two stages: first tally up each subscription's total number of sessions this month, then use that to shortlist subscriptions with fewer than 5 — these may be at risk of cancellation.	SELECT / JOIN / aggregation
2	A satellite previously marked as retired has just been reactivated after a successful repair maneuver. Update its status to reflect this.	DDL / DML / security
3	For each subscription, looking at its sessions in date order, show what the latency was on the reading immediately before the current one — this helps catch any sudden, unexplained connection quality drop.	Window function
4	Calculate a combined usage score per customer using their total data consumption and session count as an intermediate result, then use that to rank countries by overall network engagement.	CTE / analytic
5	Identify any subscription that has never logged a single usage session — a red flag for the customer success team to investigate.	SELECT / JOIN / aggregation
6	After a recent billing system cleanup, a few usage logs may now point to a subscription ID that no longer exists. Track down these orphaned records.	LEFT JOIN / integrity
7	Support agents constantly search usage logs using a specific subscription's ID to pull up its full session history instantly, but this lookup has started to feel sluggish now that the log table has grown into the tens of thousands. What would you do about it?	Indexing
8	A billing system recalibration means every monthly fee under one specific plan type needs to be adjusted by a fixed percentage. Apply this change to just that plan type's subscriptions.	DDL / DML / security
9	For each plan type, build a running count of new subscriptions signed up over time, ordered by start date, to visualize how each plan has grown.	SELECT / JOIN / aggregation
10	List all satellites that were launched in the same year as the satellite with the most subscribers.	Aggregation
11	Within each country, order customers by their total data usage, highest first, to identify the network's heaviest users market by market.	Aggregation
12	Produce a single combined list of every satellite and every subscription together — including satellites with zero subscribers and any subscription somehow missing a satellite link — so nothing gets left out of the audit.	SELECT / JOIN / aggregation
13	Prepare a network report listing every subscription along with the customer's name and the satellite it's connected to, all on a single line.	SELECT / JOIN / aggregation
14	For each plan type, calculate the total monthly revenue it generates, and highlight any plan type contributing less than 10% of total revenue — a pricing strategy question worth raising.	Revenue / aggregation
15	Group all usage logs by subscription, and flag subscriptions with fewer than 3 total sessions recorded this month — these customers may be about to churn.	SELECT / JOIN / aggregation
16	For a satellite currently under review, work out how many days passed between its launch date and its very first maintenance record.	Date / string

No.	Business requirement	Primary SQL concept
17	For the customer support dashboard, combine each customer's name and country into a single readable label, separated by a comma.	SELECT / JOIN / aggregation
18	The engineering dashboard constantly filters by satellite and maintenance date together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?	Indexing
19	Some satellites may not be serving a single active subscription right now. Identify exactly which ones, since this is unused capacity worth flagging.	LEFT JOIN / integrity
20	The quarterly network health report needs a single consolidated result: for every satellite, the number of subscriptions it serves, their average session latency, the total downtime from maintenance, and a clear flag for any satellite whose average latency has crossed an unacceptable threshold.	SELECT / JOIN / aggregation
21	Group maintenance records by satellite and identify which satellites have had more than 10 recorded incidents — these may need to be prioritized for early retirement.	Aggregation
22	Flag each satellite as 'Overloaded', 'Balanced', or 'Underutilized' based on how many subscriptions are currently linked to it relative to its typical capacity.	CTE / analytic
23	Count how many active subscriptions currently exist under each plan type.	Revenue / aggregation
24	Split all customers into three roughly equal usage tiers based on total data consumed, to help design three different targeted upsell campaigns.	CTE / analytic
25	Based on session latency, tag each usage log as 'Excellent', 'Acceptable', or 'Poor' connection quality, to help support agents quickly diagnose complaints.	CASE
26	First tally each country's total customer count, then bring in revenue details to see which countries are generating the most revenue per customer.	Revenue / aggregation
27	The operations director wants one number for the board meeting: exactly how many customer sessions have been logged across the entire network so far.	Aggregation
28	A data-entry mistake caused an entire batch of customers to be logged under the wrong country. Correct this in one sweep rather than customer by customer.	DDL / DML / security
29	After several rounds of database changes, nobody quite remembers what was set up to speed up lookups on the usage logs table. Check what's currently in place. show index from usagelogs;	Indexing
30	A usage session arrives logged against a subscription ID that was never created in the system. Explain what should happen to this record and why it must not slip through quietly.	SELECT / JOIN / aggregation
31	The scheduling team wants to know which day of the week sees the highest data usage, to plan network maintenance windows around quieter periods.	Aggregation
32	A bulk import left several ground station names with accidental leading or trailing spaces. Clean these up before they go into the network status report.	DDL / DML / security
33	A batch of very old usage logs from a discontinued legacy billing cycle is cluttering the database and is no longer needed for any analysis. Remove exactly those records.	DDL / DML / security
34	Pull every usage session recorded during the month of March this year, to study seasonal demand patterns.	Date / string
35	Label each usage session as 'Light' (under 5 GB), 'Moderate' (5-20 GB), or 'Heavy' (above 20 GB), so the network team can understand consumption patterns at a glance.	CASE
36	Calculate the average downtime per maintenance event, to understand how disruptive a typical incident really is.	Aggregation
37	A contractor's engagement with the network operations team has just ended, and their access to the database needs to be fully withdrawn with immediate effect.	DDL / DML / security

No.	Business requirement	Primary SQL concept
38	For the infrastructure report, work out the average orbit altitude across the entire satellite fleet.	Date / string
39	The network engineering team keeps asking for the same list — every satellite with its total count and total downtime. Set up something they can pull directly, without you explaining the underlying tables every time they ask.	View
40	For each subscription, show what the very next session's data usage will be, placed right alongside the current one, to help spot a pattern building up across consecutive sessions.	Window function
41	For each satellite, smooth out momentary reporting glitches by calculating an average latency using only the current reading and the two immediately before it.	Window function
42	Find the lowest latency ever recorded across the network — this benchmark could be used in the next marketing campaign.	Aggregation
43	Cross-check satellites against maintenance activity to identify any satellite that has never had a single maintenance record — either remarkably reliable or worryingly unmonitored.	LEFT JOIN / integrity
44	Group all customer signups by the exact quarter they joined in, to compare growth across different phases of the network's rollout.	Date / string
45	An engineer accidentally types a satellite's orbit altitude as a negative number. How do you make sure this kind of entry is rejected the moment it's submitted?	DDL / DML / security
46	Some customer names were typed with inconsistent capitalization during onboarding. Produce a clean, uniformly capitalized list for the billing system.	SELECT / JOIN / aggregation
47	For each customer, work out how many days have passed since they signed up — this determines who's eligible for the one-year loyalty discount.	Date / string
48	List every subscription along with its most recent usage session, making sure subscriptions that haven't logged any usage yet still appear on the list rather than disappearing.	LEFT JOIN / integrity
49	Across the entire network, order every usage session by data consumed to surface the season's heaviest sessions, making sure that if two sessions used the exact same amount of data, neither one gets unfairly skipped in the ordering.	Window function
50	Sort each satellite's launch date into 'Legacy' (before 2021), 'Mid-Generation' (2021-2023), or 'New Generation' (after 2023), to help engineers prioritize fleet upgrades.	CASE
51	Find the single highest data usage recorded in one session — this could be a business customer running a large upload job worth understanding better.	Aggregation
52	The system currently lets someone log a session latency of -40 milliseconds by mistake. What change would permanently make this impossible going forward?	DDL / DML / security
53	Pull every usage log belonging specifically to the one subscription that recorded the highest singlesession data usage in the system.	SELECT / JOIN / aggregation
54	Find every plan type that has at least one subscriber in a country where the average latency is above 90 milliseconds.	CTE / analytic
55	Build an intermediate satellite-by-satellite average latency figure, then use it to flag every satellite performing worse than the network-wide average.	CASE
56	Track down which satellite is linked to the subscription that recorded the single highest data usage anywhere in the entire dataset.	SELECT / JOIN / aggregation
57	The regional expansion team wants to flag any ground station name that's unusually long (more than 30 characters) so it fits the map interface properly.	Date / string
58	Work out how many distinct countries the customer base currently spans.	Aggregation
59	Identify every satellite that was launched more than five years ago — these units are approaching the end of their expected operational life.	Date / string

No.	Business requirement	Primary SQL concept
60	Break this into two steps: first find each subscription's most recent session date, then use that to identify which subscriptions have gone quiet in the last 30 days.	Date / string
61	The billing supervisor needs the ability to update subscription fee records directly, unlike the rest of the support staff. Set up an access level that reflects this difference.	DDL / DML / security
62	The finance team wants to know, in one number, the total monthly revenue currently being generated from all active subscriptions.	Revenue / aggregation
63	A new customer support agent is joining the team and should only be able to look at customer and subscription data, never change any of it. Set up their access accordingly.	DDL / DML / security
64	Going forward, the network wants to record which manufacturer built each satellite. Adjust the satellite records so this detail can be captured.	DDL / DML / security
65	A support agent tries to save a new subscription without linking it to a customer. What should prevent that incomplete record from being saved?	DDL / DML / security
66	Count the subscriptions currently linked to each satellite, and shortlist only the satellites carrying more than 20 subscribers — these may be nearing capacity.	Aggregation
67	Two onboarding agents in different regions accidentally try to register the same satellite name twice during a fleet update. What should stop this from happening?	DDL / DML / security
68	For every satellite, surface the single highest-usage subscription linked to it without losing any of that satellite's other subscription rows in the process — every subscription should still show up in the result.	Window function
69	Identify which countries have an average session data usage above 20 GB — these markets may need additional network capacity.	Aggregation
70	Identify every subscription belonging to whichever single satellite currently has the highest number of subscribers.	Aggregation
71	The finance dashboard wants something it can plug straight into its reporting tool that always reflects each plan type's current total monthly revenue, without re-deriving it manually every cycle.	Revenue / aggregation
72	For a specific customer flagged for unusual activity, pull all their sessions that occurred between midnight and 5 AM.	Date / string
73	For every ground station, list out all the countries whose customers are being served through that station's coverage footprint (matched via country).	SELECT / JOIN / aggregation
74	For each subscription, flag any of its own sessions where the data usage is noticeably higher than that same subscription's own typical usage — a possible sign of a compromised account or a new use case.	Window function
75	The customer support team needs a simplified, always-ready snapshot showing each customer's name, plan type, assigned satellite, and most recent session, without agents having to reconstruct the full picture from scratch on every call.	Window function

31. Project Summary / Portfolio Page

Portfolio field	Value
Project	OrbitLink Satellite Internet Network
Type	SQL Database Management & Analytics
Database	MySQL / SQL
Tables	6
Records	20,000

Portfolio field	Value
Usage sessions	18,200
Customers	700
Subscriptions	700
Maintenance events	335
Satellites	40
Ground stations	25
Core strengths	Relational modeling, joins, aggregation, CTEs, window functions, data quality, indexing, views, access control
Prepared by	Prativa Baliarsingh

Portfolio positioning: A source-accurate, end-to-end SQL case study showing how operational requirements become relational structures, analytical queries, reusable reporting objects, validation rules, performance improvements, and business insights.

Primary source files: orbitlink_full.sql • OrbitLink_Satellite_Project.pdf • EER diagram sql project 3.mwb